COMPILING FLAGCALC AND EXPLORE FROM SOURCE

PETER GLENN

1. Installing JetBrains CLion as a means to compile from source

(1) Install JetBrains Toolbox
- Either
  - in a web browser download from `https://www.jetbrains.com/toolbox-app/` or
  - at the bash shell command line run `wget -c "https://jetbrains.com"`
- Either
  - use file manager: double-click the downloaded tar.gz file in "Downloads" and extract to `/opt`, or
  - use the command line `sudo tar -xzf jetbrains-toolbox-⟨version number⟩.tar.gz -C /opt`
  - Note that the version number can be found by running `ls /Downloads/jet*.tar.gz`
- Launch the app `/opt/jetbrains-toolbox-⟨version number⟩/bin/jetbrains-toolbox`
- In the app window, click on CLion and if desired, PyCharm, to download those; while downloading, log in to JetBrains or create a free account (note they offer free CLion and PyCharm licenses for non-commercial use; read their contract to verify)
- If all goes well, the Jetbrains toolbox will launch and appear in your status bar system-wide upon next boot of your machine

(2) Open CLion, pin it to your task bar if desired, choose "Clone a project from Git"; enter the Github URL:

`https://github.com/corralledcode/flagcalc.git`

(3) *Optional:* Repeat, using the menu items "flagcalc... Project from version control" of CLion, if desired for the GUI front-end to flagcalc, "Explore":

`https://github.com/corralledcode/explore.git`

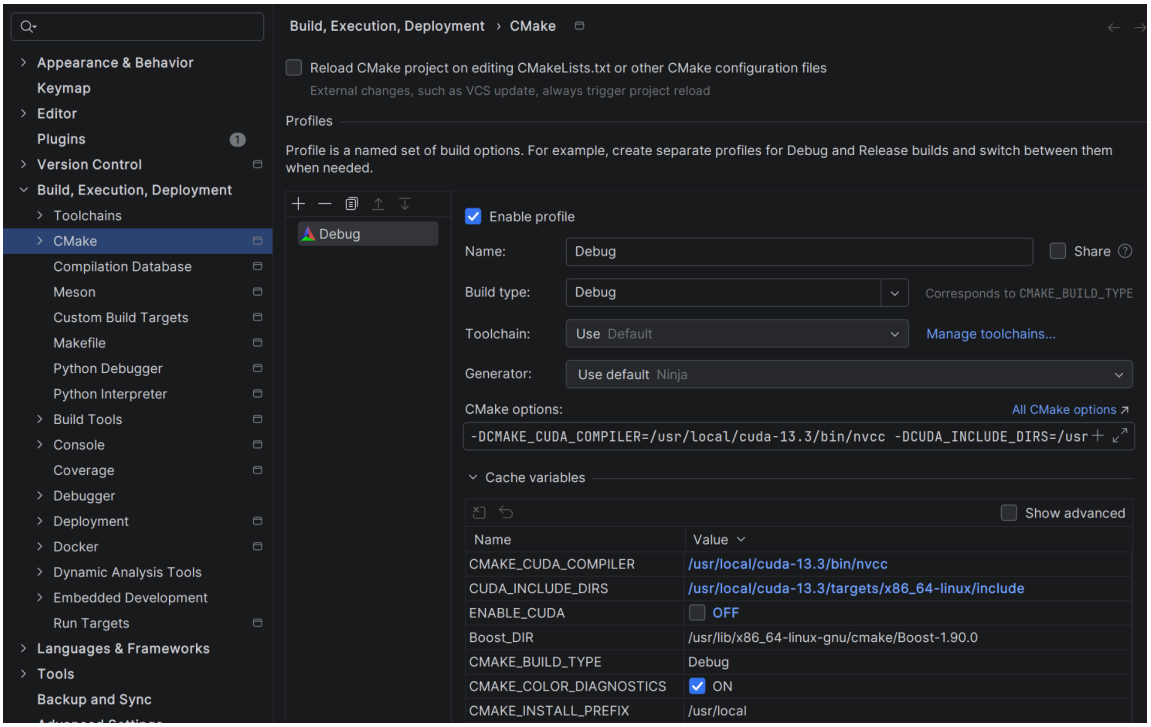
(4) You may verify that CLion builds flagcalc with errors. Proceeding, install Boost
- `sudo apt update && sudo apt install libboost-all-dev -y`

(5) Next, install PyBind11
- `sudo apt install -y build-essentials cmake python3-dev python3-pip`
- `sudo apt install -y python3-pybind11 pybind11-dev`

Date: June 15, 2026.

(6) Reload CLion (or get used to clicking `File... Reload CMake Project`) and try clicking the hammer icon in the top right of the window to build all.
(7) Aside: building with nvcc, that is, with NVidia GPU code
- Please note this code is in its infancy, and this is not a complete instruction set below
- Follow the instructions on the NVidia page
  `https://developer.nvidia.com/cuda-downloads`
- Also run
  - `sudo apt install nvidia-cuda-toolkit`
- run `nano /.bashrc` and add at the end of the file (replacing `cuda` with e.g. `cuda-13.3`)
  - `export PATH="/usr/local/cuda/bin:$PATH$"`
  - `export LD_LIBRARY_PATH="/usr/local/cuda/lib64:$LD_LIBRARY_PATH$"`
- essential in my experience now to reboot (`sudo reboot` from the terminal), in order for the next command to return a version number:
  - `nvcc --version`
- In CLion under "File.. Settings.. Build, Execution, Deployment, CMake options" add the options
  - `-DCMAKE_CUDA_COMPILER=/usr/local/cuda-13.3/bin/nvcc`, and
  - `-DECUDA_INCLUDE_DIRS=/usr/local/cuda-13.3/targets/x86_64-linux/include`
  - `-DENABLE_CUDA:BOOL=OFF`

  which will appear in "Cache variables" immediately below (click the caret to expand). Note after all the hard work of installing CUDA, the instructions here are just to leave it off, but feel free to experiment.



(8) Build and run flagcalc. Click the green play button in the top right, but first to make the output meaningful, click Edit configurations, and enter program arguments such as
   - `-r 6 8 200 -f all -i sortedverify -v i=minimal3.cfg`

   "Working directory" should be .../`CLionProjects/flagcalc/testgraph` (since that is where e.g. `minimal3.cfg` is located)

(9) Now add all the shell scripts from the folder `/CLionProjects/flagcalc/scripts` via:
   - clicking the plus in the top left of the Edit configurations dialog box
   - inputting a name for each shell script
   - browsing to its exact location
   - also specifying the working directory .../`CLionProjects/flagcalc/testgraph`
   - to be agreeable with the "Explore" GUI, it is required to specify an Environment variable for each script as in:
     - `PTH=../cmake-build-debug`

(10) Note the project directory in CLion's left-hand pane, where you can double-click a shell script to open it up. CLion does not syntax-highlight .sh files (the key challenge to the naked eye being matching left and right parentheses in deeply-nested queries), but it does .dat files (which is the naming convenction for "stored procedures" such as ...`scripts/storedprocedures.dat`). This is remedied by the syntax-highlighting built in to "Explore", the other main project and helper project

to flagcalc, on the Corralled Code Github page `http://github.com/corralledcode/explore`.

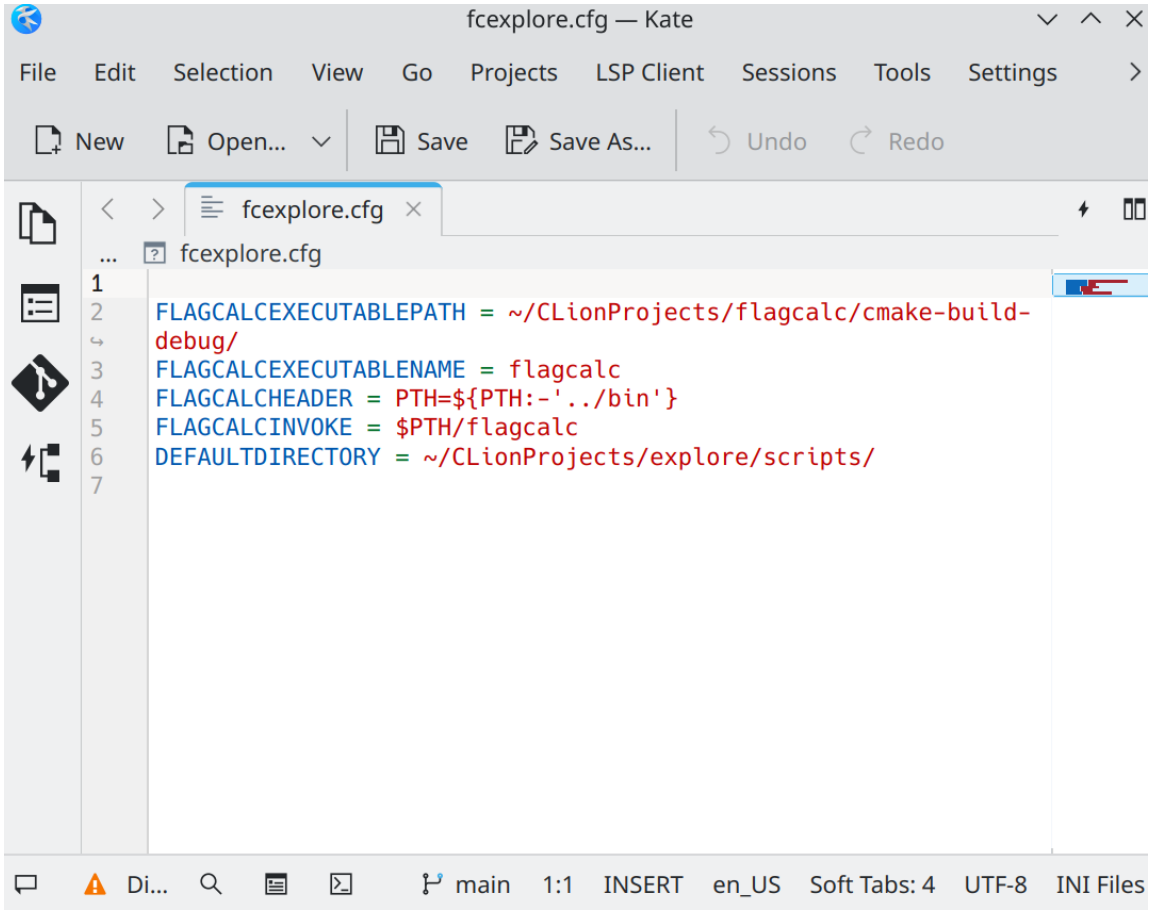
We now compile Flagcalc's GUI "Explore".

(1) Install Qt6
   - `sudo apt install build-essential libgl1-mesa-dev cmake -y`
   - `sudo apt install qt6-base-dev -y`
   - `sudo apt install qt6-tools-dev-tools -y`
   - `sudo apt install qt6-tools-dev -y`

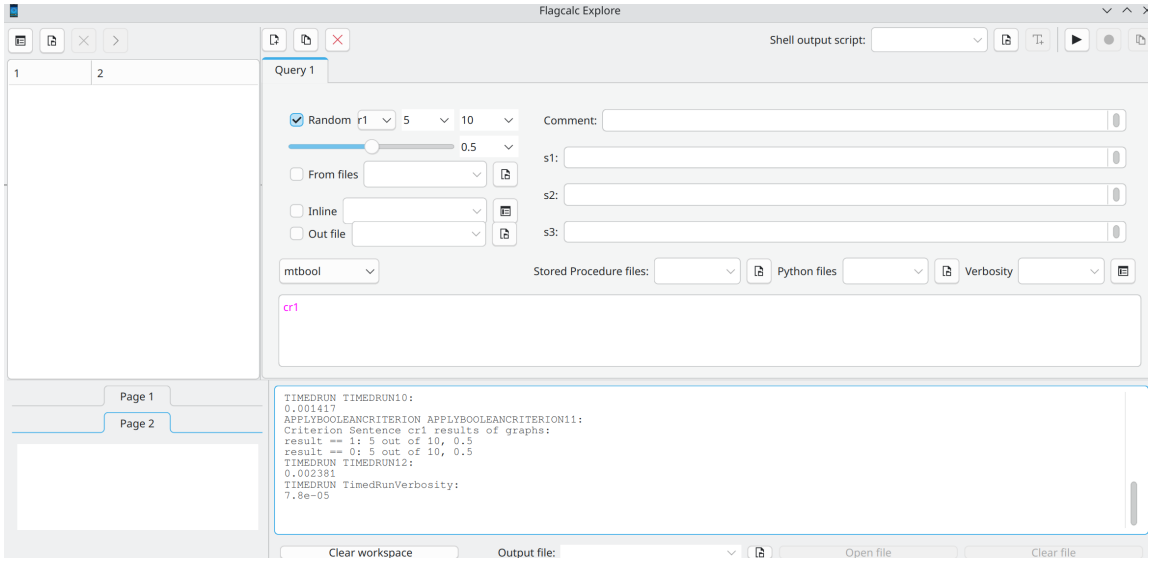
(2) Assuming you haven't already, use the CLion menu to pull a new project from a repository, using the URL `https://github.com/corralledcode/explore.git`.

(3) Reload the project and click build or click the green Run button

(4) Note there is a file in the `.../CLionProjects/explore` called `fcexplore.cfg` seen in the screenshot below



(5) If Explore runs, try a simple query as below to be sure it correctly found the command line tool flagcalc



(This query uses the measure/criterion `cr1`, namely, true/false as to the graph being connected or not; the result shows exactly half of the ten random graphs chosen are connected.)